

OpenModelLab: A Framework for AI Model Genome Extraction and Reproducible Benchmark Profiling

Version: 0.4.0 **Author:** Ajay Govind S **Repository:** <https://github.com/ajaygovinds/OpenModelLab>
License: MIT

Abstract

The rapid growth of publicly available artificial intelligence models has created a growing need for systematic methods to understand, characterize, and compare them. Although model repositories provide access to thousands of pretrained models, information about architecture, hardware requirements, memory consumption, and runtime performance is often distributed across model cards, configuration files, documentation, and independent benchmarks.

This paper presents **OpenModelLab**, an open-source framework for generating standardized **Model Genome Reports** and **Benchmark Reports** for AI models. OpenModelLab combines static model inspection with hardware-aware runtime profiling to capture model architecture, tokenizer characteristics, parameter count, hardware configuration, precision, model fingerprints, latency, throughput, memory usage, and batch scaling behavior.

The current implementation supports Hugging Face Transformers models and produces machine-readable reports that can be used for model comparison and further analysis. Experimental evaluation across multiple Transformer architectures demonstrates that OpenModelLab can profile models ranging from approximately 11.7 million to 335 million parameters and expose measurable differences in runtime behavior, memory consumption, and batch scaling.

A live Hugging Face demonstration further demonstrates that models beyond the initial evaluation set can be analyzed through the framework.

1. Introduction

The number and diversity of publicly available AI models continue to increase rapidly. Model repositories such as Hugging Face provide convenient access to pretrained models, but comparing models remains difficult when evaluation requires more than parameter count.

Users and developers may need to answer questions such as:

- What architecture does a model use?
- How many parameters does it contain?
- What are its architectural characteristics?
- What hardware is required to execute it?
- How much GPU memory does inference consume?
- What is its inference latency?
- What throughput can it achieve?
- How does performance change with batch size?

These characteristics are typically available in different locations and are often measured under different experimental conditions.

OpenModelLab approaches an AI model as a measurable computational artifact. It introduces a structured representation called a **Model Genome**, combined with runtime benchmark information, to provide a common foundation for model inspection and comparison.

2. Problem Statement

There is no simple standardized representation that combines the identity, architecture, execution environment, and measured runtime characteristics of an AI model.

A useful model description should include at least:

- Model identity
- Architecture
- Configuration
- Parameter count
- Tokenizer characteristics
- Hardware environment
- Precision
- Runtime behavior
- Memory consumption
- Batch scaling behavior

OpenModelLab addresses this problem through two complementary artifacts:

1. **Model Genome Report** — describes the static and execution characteristics of a model.
 2. **Benchmark Report** — records measured runtime behavior under a defined environment.
-

3. OpenModelLab

OpenModelLab is implemented as a Python-based inspection and benchmarking framework.

The current workflow is:

Hugging Face Model

Model Loader

Model Genome
Architecture
Parameters
Configuration
Tokenizer
Hardware
Precision
Fingerprint

Benchmark
Latency
Throughput
Memory
Batch Scaling

JSON / CSV / HTML / Charts

The command-line interface provides model genome generation and model comparison functionality.

A lightweight Hugging Face Space provides an interactive demonstration of the framework.

4. Model Genome

The Model Genome represents measurable characteristics of an AI model in a structured format.

4.1 Model Information

The current implementation extracts:

- Model name
- Model architecture
- Model type
- Hidden size
- Number of hidden layers
- Number of attention heads
- Intermediate size
- Maximum position embeddings
- Parameter count
- Data type
- Framework
- License

For example:

Architecture: BertModel
Layers: 12
Hidden Size: 768
Attention Heads: 12

4.2 Tokenizer Information

Tokenizer inspection includes characteristics such as:

- Vocabulary information
- Tokenizer configuration
- Special tokens
- Maximum sequence characteristics

4.3 Hardware Information

The hardware inspector records:

- PyTorch version
- CUDA availability
- CUDA version
- GPU name
- Visible GPU memory
- Free GPU memory
- Compute capability

This allows benchmark results to retain information about the execution environment.

4.4 Model Fingerprinting

OpenModelLab generates architecture-oriented fingerprints that provide a compact representation of model structure.

For example:

`bert-6L-384H-12A`

can represent:

- 6 layers
- 384 hidden dimensions
- 12 attention heads

Such fingerprints can support future model search, clustering, and comparison functionality.

5. Benchmark Framework

OpenModelLab currently provides four runtime profiling components.

5.1 Latency

Latency is measured using multiple inference runs after warm-up.

The current implementation records:

- Average latency
- Minimum latency
- Maximum latency
- Standard deviation
- Number of warm-up runs
- Number of benchmark runs

The default benchmark uses five warm-up runs followed by twenty measured runs.

CUDA synchronization is performed around GPU measurements to reduce asynchronous execution effects.

5.2 Memory

The memory inspector records:

- Process RAM before inference
- Process RAM after inference
- GPU allocated memory
- Peak GPU allocated memory

5.3 Throughput

Throughput is calculated as:

`samples / total inference time`

The current benchmark performs fifty inference runs after warm-up.

5.4 Batch Scaling

OpenModelLab evaluates batch sizes:

1
2
4
8

For each batch size it records:

- Batch latency
- Samples per second

This provides an initial view of how model execution scales under increasing batch sizes.

6. Experimental Methodology

The initial GPU evaluation was performed using an NVIDIA A100-SXM4-40GB configured as a **1g.5gb MIG instance**.

The primary environment was:

Component	Configuration
GPU	NVIDIA A100-SXM4-40GB MIG 1g.5gb
Visible GPU Memory	~4.86 GB
Compute Capability	8.0
CUDA	12.4
PyTorch	2.6.0+cu124
Python	3.11

Models were loaded using Hugging Face Transformers and executed on CUDA.

The benchmark uses a maximum sequence length of 128 tokens. Results represent the specific execution environment and benchmark configuration rather than universal performance characteristics.

7. Initial Model Evaluation

Four models were initially evaluated:

Model	Parameters	Layers	Hidden Size
BERT	109.48M	12	768
DistilBERT	66.36M	6	768
MiniLM	22.71M	6	384
MobileBERT	24.58M	24	512

OpenModelLab generated Model Genome and Benchmark reports for each model.

GPU Results

Model	Latency (ms)	Throughput (samples/s)	Peak GPU Memory (MB)
DistilBERT	3.076	345.672	263.6
MiniLM	3.103	323.450	95.9
BERT	5.990	167.802	428.1
MobileBERT	27.134	36.643	103.1

The measurements demonstrate that parameter count alone does not determine runtime behavior. MobileBERT, for example, has substantially fewer parameters than BERT but exhibited substantially higher latency in this benchmark.

8. CPU vs GPU Evaluation

The initial evaluation also compared CPU and GPU execution.

Model	CPU Latency (ms)	GPU Latency (ms)	CPU Throughput	GPU Throughput
BERT	74.689	5.990	15.012	167.802
DistilBERT	14.260	3.076	79.182	345.672
MiniLM	6.210	3.103	112.906	323.450
MobileBERT	83.750	27.134	15.253	36.643

The comparison demonstrates the impact of execution hardware on measured runtime performance.

BERT showed a substantial reduction in latency when executed on the GPU. DistilBERT achieved the highest GPU throughput among these four models.

These results should be interpreted as measurements of the tested environment rather than general hardware rankings.

9. Cross-Architecture Validation

To determine whether OpenModelLab could analyze models beyond the initial four-model evaluation set, additional models were tested through the public Hugging Face demonstration.

The evaluation expanded the tested parameter range from approximately **11.7M to 335M parameters**.

Additional models included:

Model	Parameters	Architecture
ALBERT-base-v2	11.68M	AlbertModel
RoBERTa-base	124.65M	RobertaModel
BERT-large-uncased	335.14M	BertModel

The results were generated automatically by the live OpenModelLab Hugging Face application.

10. Live Hugging Face Evaluation

The OpenModelLab Hugging Face Space provides an interactive interface where a user can supply a Hugging Face model identifier and request profiling.

For example, the live evaluation of `google-bert/bert-large-uncased` produced:

Metric	Result
Parameters	335.14M
Architecture	BertModel
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition MIG 2g.48gb
CUDA	13.0
Latency	6.583 ms
Throughput	151.8 samples/s
Peak GPU Memory	1288.8 MB

Batch scaling produced:

Batch Size	Latency (ms)	Samples/s
1	6.583	151.908

Batch Size	Latency (ms)	Samples/s
2	7.767	257.505
4	7.752	515.987
8	9.115	877.678

The live evaluation demonstrates that the framework can be applied to models that were not part of the original local benchmark set.

Because the Hugging Face Space uses a different GPU and software environment from the original A100 experiment, these measurements are **not directly comparable** to the A100 results. They instead demonstrate portability of the profiling workflow across execution environments.

11. Observations

Several observations emerge from the experiments.

11.1 Parameter count is not sufficient

Models with fewer parameters do not necessarily achieve lower latency.

MobileBERT provides a clear example: despite its smaller parameter count, its measured latency was substantially higher than BERT in the tested environment.

11.2 Architecture matters

Layer count, hidden dimensions, attention configuration, and implementation details all influence runtime characteristics.

Therefore, parameter count should be treated as one characteristic rather than a complete indicator of computational cost.

11.3 Hardware matters

The same model can exhibit substantially different performance depending on execution hardware, CUDA environment, software stack, and available GPU resources.

11.4 Batch size changes throughput

Increasing batch size generally increased aggregate samples per second in the evaluated experiments, although latency also changed with batch size.

This makes batch scaling an important part of practical model characterization.

12. Reproducibility

OpenModelLab stores benchmark measurements together with execution metadata.

A benchmark report includes:

- Tool version
- Schema version
- Timestamp
- Hardware
- CUDA version
- Latency configuration
- Throughput configuration
- Memory measurements
- Batch scaling results

This allows benchmark results to be preserved as machine-readable artifacts rather than only being reported as isolated numbers.

However, exact numerical reproduction can be affected by:

- GPU model
- MIG configuration
- CUDA version
- PyTorch version
- Transformers version
- Sequence length
- Batch size
- System load
- Model implementation

Therefore, OpenModelLab treats benchmark results as **environment-specific measurements**.

13. Limitations

The current implementation has several limitations:

- Evaluation is currently focused primarily on Hugging Face Transformers models.
 - Benchmarking is currently inference-oriented.
 - Sequence lengths are currently limited by the benchmark configuration.
 - Distributed and multi-GPU benchmarking is not yet implemented.
 - Benchmark results are not yet stored in a centralized public database.
 - No statistical confidence intervals are currently reported.
 - The current benchmark suite does not cover all model families or generative workloads.
 - Performance measurements from different hardware environments should not be treated as directly comparable without controlled experimental conditions.
-

14. Future Work

Future versions of OpenModelLab may explore:

- Large-scale public model benchmark databases
- Automated hardware suitability prediction
- Model recommendation
- Model performance leaderboards
- Additional model families
- Generative model benchmarking
- Multi-GPU and distributed evaluation
- Quantization-aware profiling
- CPU/GPU/accelerator comparison
- Interactive visualization dashboards
- Hugging Face integration
- Standardized model metadata schemas

The longer-term objective is to establish a reusable ecosystem for describing and comparing AI models as measurable computational artifacts.

15. Conclusion

OpenModelLab presents an initial open-source framework for standardized AI model inspection and runtime profiling.

The Model Genome concept combines architectural, tokenizer, hardware, precision, and fingerprint information with measured runtime characteristics. The benchmark framework complements this static information with latency, throughput, memory, and batch-scaling measurements.

Experiments across multiple Transformer architectures demonstrate that OpenModelLab can analyze models spanning approximately 11.7M to 335M parameters and can operate across different GPU execution environments.

The project is intended as an evolving foundation rather than a final benchmarking standard. Future work will focus on expanding model coverage, improving reproducibility, building a larger benchmark corpus, and developing methods for automated model and hardware comparison.

16. Availability

GitHub: <https://github.com/ajaygovinds/OpenModelLab>

Hugging Face Demo: <https://huggingface.co/spaces/ajaygovind/OpenModelLab>

Research / Zenodo: <https://doi.org/10.5281/zenodo.21763818>

Appendix A — Example Model Genome

An OpenModelLab genome contains structured information similar to:

```
{
  "model_name": "sentence-transformers/all-MiniLM-L6-v2",
  "architecture": "BertModel",
  "parameter_count": 22700000,
  "hidden_size": 384,
  "num_hidden_layers": 6,
  "num_attention_heads": 12,
  "framework": "transformers"
}
```

The exact report schema may evolve with future OpenModelLab versions.

Appendix B — Example Benchmark

A benchmark report contains structured measurements such as:

```
{
  "latency": {
    "average_ms": 3.103
  },
  "throughput": {
    "samples_per_second": 323.45
  },
  "memory": {
    "gpu_peak_allocated_mb": 95.9
  }
}
```

These machine-readable artifacts enable subsequent comparison, visualization, and analysis.

AI Assistance Disclosure

Generative AI tools were used during software development and documentation preparation. All implementation, experiments, measurements, and reported results were performed and verified by the author.